Attention Inference Algorithm

HJ

November 7, 2026

1 Introduction

These notes survey some papers on attention inference.

Ideally, an attention kernel computes the current tile in on-chip SRAM while all other tiles (including the KV cache) are staged in HBM. Each paper attacks one side of this picture:

- **FlashAttention** handles the compute side: an SRAM-aware kernel that keeps the working tile on chip and avoids materializing the full $N \times N$ attention matrix in HBM.
- **PagedAttention** handles the memory side: the KV cache is stored as non-contiguous pages behind a block table, removing fragmentation. Note that vLLM's scheduler is still first-come-first-served (FCFS); paging is PagedAttention's own contribution, not scheduling.
- **RadixAttention** builds on PagedAttention's paging and adds two things: prefix reuse through a radix tree with LRU eviction, and cache-aware scheduling (longest-prefix-first over the KV-cache tree) that replaces FCFS.

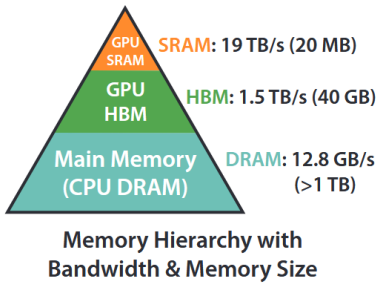


Figure 1: Memory hierarchy of CPU and GPU (SRAM vs HBM).

2 FlashAttention

2.1 Compute-bound & Memory-bound Operators

- **Compute-bound**: High arithmetic intensity. GEMM (matrix multiply) — moves $O(n^2)$ data but performs $O(n^3)$ FLOPs, so arithmetic intensity grows with n.
- **Memory-bound**: Runtime is limited by memory bandwidth. softmax / dropout / element-wise ops — they move $O(n)$ data while doing only $O(n)$ FLOPs, so they spend most of their time waiting on memory.

Attention is memory-bound: $S = QK^\top$ and $P = \text{softmax}(S)$ must be written to and read back from HBM. FlashAttention's contribution is keeping S, P in SRAM instead of materializing them in HBM, thereby reducing HBM traffic.

2.2 SRAM-awareness Tiling in v1

Instead of materializing S and P in HBM, FlashAttention tiles Q, K, V into blocks that fit in on-chip SRAM, computes each block's attention statistics there, and only writes the final output O back to HBM. This reduces HBM traffic — the tiles still move between SRAM and HBM, but the huge $N \times N$ intermediates never do.

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.

2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Figure 2: FlashAttention v1 SRAM tiling: Q, K, V blocks are computed on chip.

- **Why 4?** A tile of Q, K, V, and O must all sit in SRAM at once, so each block is bounded by roughly $M/4d$ (with M the SRAM size and d the head dimension).
- **Why $\min(\cdot, d)$?** Without a cap, $B_r B_c \approx (M/4d)^2$; keeping the score tile S within a quarter of SRAM then requires $(M/4d)^2 < M/4$, i.e. $M < 4d^2$, which often fails.

2.3 Reduce Axis Q in Forward

For the outer reduction axis, v1 uses a KV scheme, while v2 uses Q scheme. The new scheme has two advantages:

- **Performance**: fewer HBM accesses — the output accumulator stays on chip and is written back once instead of once per K, V block.
- **Interpretation**: decoding-friendly — decoding only needs the single row of Q for the new token.

5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**

Figure 3: FlashAttention v1 forward algorithm (KV-split).

3: **for** $1 \le i \le T_r$ **do**
4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
6: **for** $1 \le j \le T_c$ **do**
7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
11: **end for**
12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
14: Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
15: Write L_i to HBM as the i-th block of L.
16: **end for**

Figure 4: FlashAttention v2 forward algorithm (Q-split).

Triton puzzle 9 the Scalar FlashAttention can be seen as decoding once. is a good exercise for understanding how the v2 scheme works. Notation used below: S is the accumulator over the whole row, s is the local accumulator for the current tile.

2.4 Reduce Axis KV in backward

v2 still uses the KV-split scheme in the backward pass. The reason lies in the reduction axes: dK and dV share a reduction axis, while dS has an independent one.

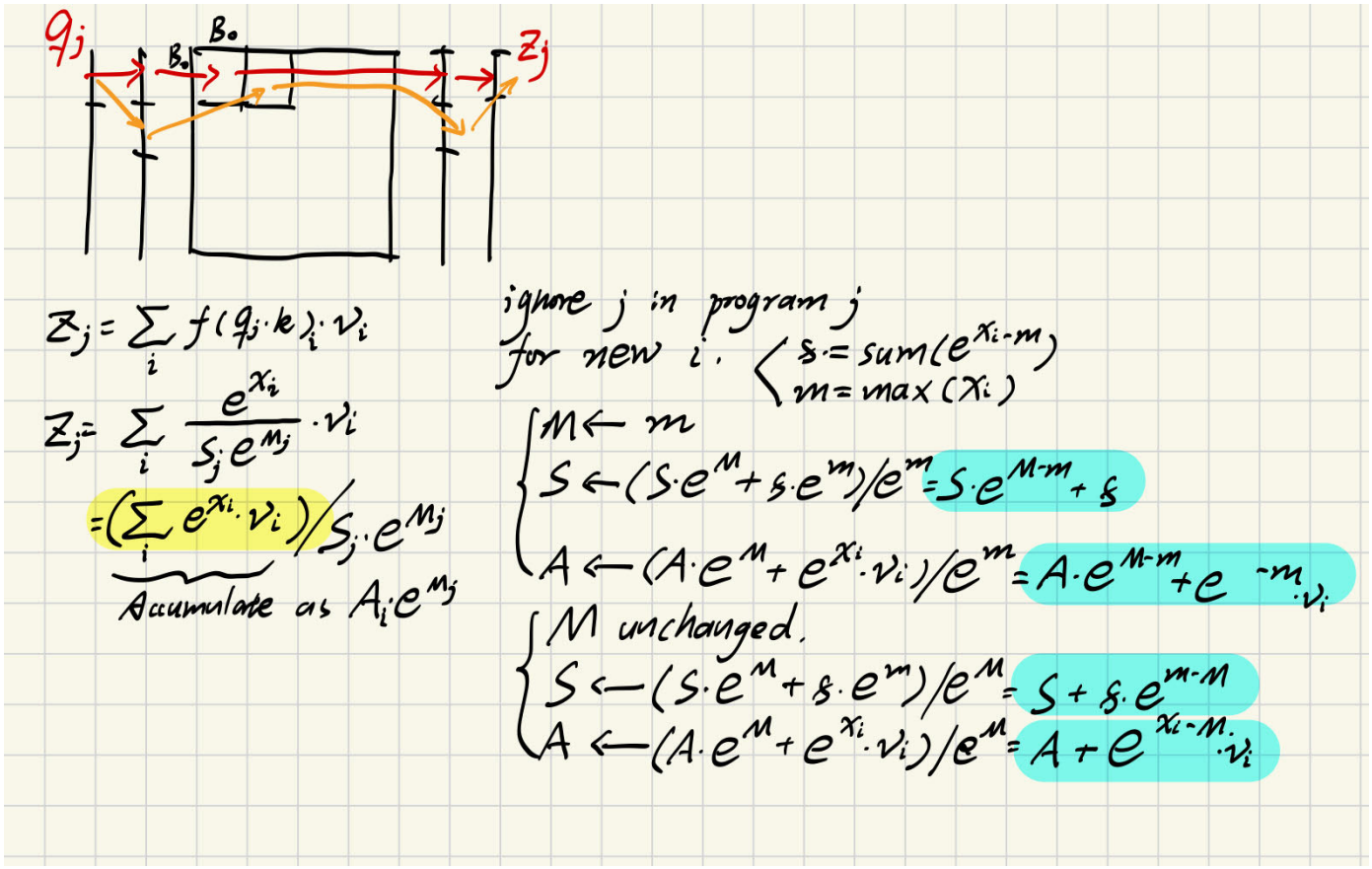


Figure 5: Triton puzzle 9 (FlashAttention forward).

- An outer reduction axis can be accumulated on chip and written back once
- An inner reduction axis changes at every step and must be written to HBM each time.

Hence dK, dV are placed on the outer loop and dS on the inner loop, minimizing HBM traffic.

Anyway, these are all about tradeoff.

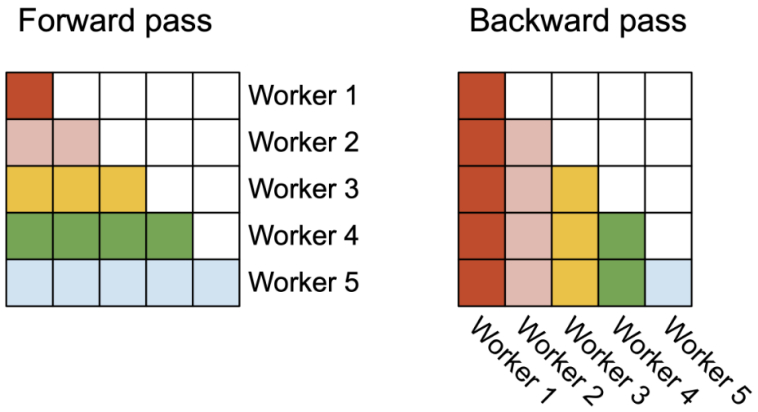


Figure 6: FlashAttention v2 backward reduce-axis scheme (dK, dV outer, dS inner).

2.5 Causal Mask Minor Trick

Because the causal mask adds $-\infty$ to masked positions, slightly fewer than half of the tiles can be skipped entirely (the strictly upper-triangular tiles), and only the diagonal tiles actually need the mask applied.

3 PagedAttention

TODO? Maybe won't do since I already take notes by zotero, and I believe those who need to read the paper won't need my notes, they could and would read original paper anyway.

4 RadixAttention

TODO?

References

[1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.

[2] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023.

[3] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.

[4] Lianmin Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. NeurIPS 2024.